```r
# ==============================================================
# PROJECT 1 — CUSTOMER SEGMENTATION
# Dataset: Mall Customer Dataset (Kaggle)
# Algorithm: K-Means Clustering
# Language: R
# Author: [Olusanya Oluwatobi]
# ==============================================================


# ==============================================================
# STEP 1 — LOAD PACKAGES
# ==============================================================

# -- Install ggrepel first (required by factoextra) ----------
if (!"ggrepel" %in% rownames(installed.packages())) {
  install.packages("ggrepel",
                   repos = "https://cloud.r-project.org",
                   type  = "binary",
                   lib   = "C:/Users/olusa/AppData/Local/R/win-library/4.4")
}

# -- Install remaining packages if missing -------------------
packages <- c("tidyverse", "cluster", "factoextra", "NbClust",
              "scales", "ggplot2", "corrplot", "gridExtra",
              "knitr", "RColorBrewer", "clustertend")

missing <- packages[!packages %in% rownames(installed.packages())]
if (length(missing) > 0) {
  install.packages(missing, dependencies = TRUE)
}

# -- Load all libraries --------------------------------------
library(ggrepel)         # Required by factoextra (load first)
library(tidyverse)       # Data manipulation and visualization
library(cluster)         # Clustering algorithms (PAM, silhouette)
library(factoextra)      # Cluster visualization (fviz functions)
library(NbClust)         # Determine optimal number of clusters
library(scales)          # Scale formatting for plots
library(ggplot2)         # Advanced visualization
library(corrplot)        # Correlation matrix visualization
library(gridExtra)       # Arrange multiple plots
library(knitr)           # Neat table outputs
library(RColorBrewer)    # Color palettes
library(clustertend)     # Hopkins statistic for cluster tendency

# -- Confirm all packages loaded correctly -------------------
cat("✅ All packages loaded successfully!\n")
cat("ggrepel   :", as.character(packageVersion("ggrepel")),   "\n")
cat("factoextra:", as.character(packageVersion("factoextra")), "\n")
cat("tidyverse :", as.character(packageVersion("tidyverse")),  "\n")


# ==============================================================
# STEP 2 — IMPORT DATA
# ==============================================================

df <- read_csv("Shopping Mall Customer Segmentation Data .csv",
               show_col_types = FALSE)
glimpse(df)       # Structure + data types
head(df, 10)      # First 10 rows
summary(df)       # Summary statistics


# ==============================================================
```

```r
# STEP 3 — DATA CLEANING & PREPROCESSING
# ============================================================

cat("\n--- STEP 3: DATA CLEANING & PREPROCESSING ---\n")

# --- 3.1 Rename Columns for Clarity ---
df <- df %>%
  rename(
    CustomerID     = `Customer ID`,
    Gender         = Gender,
    Age            = Age,
    Annual_Income  = `Annual Income`,
    Spending_Score = `Spending Score`
  )

cat("✅ Columns renamed successfully!\n")
names(df)

# --- 3.2 Check for Missing Values ---
cat("\n🔎 Missing Values per Column:\n")
colSums(is.na(df))

# Remove rows with missing values (if any)
df <- df %>% drop_na()
cat("✅ Missing values handled\n")

# --- 3.3 Check for Duplicate Rows ---
duplicates <- sum(duplicated(df))
cat("\n🔁 Duplicate Rows:", duplicates, "\n")
df <- df %>% distinct()
cat("✅ Duplicates removed\n")

# --- 3.4 Encode Categorical Variable (Gender) ---
df <- df %>%
  mutate(Gender_Encoded = ifelse(Gender == "Male", 1, 0))
cat("✅ Gender encoded: Male = 1, Female = 0\n")

# --- 3.5 Detect and Handle Outliers (IQR Method) ---
detect_outliers <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val
  sum(x < lower | x > upper)
}

cat("\n📦 Outliers Detected:\n")
cat("Age:           ", detect_outliers(df$Age), "\n")
cat("Annual Income: ", detect_outliers(df$Annual_Income), "\n")
cat("Spending Score:", detect_outliers(df$Spending_Score), "\n")

# For this dataset, outliers are minimal — we keep them but cap extreme values
df <- df %>%
  mutate(
    Age            = pmin(pmax(Age, quantile(Age, 0.01)), quantile(Age, 0.99)),
    Annual_Income  = pmin(pmax(Annual_Income, quantile(Annual_Income, 0.01)),
quantile(Annual_Income, 0.99)),
    Spending_Score = pmin(pmax(Spending_Score, quantile(Spending_Score, 0.01)),
quantile(Spending_Score, 0.99))
  )
cat("✅ Outliers capped at 1st and 99th percentile\n")
```

```r
# --- 3.6 Feature Scaling (Normalization) ---
# CRITICAL for K-Means: scale all numeric features to same range
features_to_scale <- c("Age", "Annual_Income", "Spending_Score")

df_scaled <- df %>%
  mutate(across(all_of(features_to_scale), scale))

cat("✅ Features scaled (z-score normalization)\n")

# Summary after cleaning
cat("\n📊 CLEANED DATA SUMMARY:\n")
summary(df[, c("Age", "Annual_Income", "Spending_Score")])


# ============================================================
# STEP 4 — FEATURE ENGINEERING
# ============================================================

cat("\n--- STEP 4: FEATURE ENGINEERING ---\n")

df <- df %>%
  mutate(
    # Income-to-Spending Ratio: high income but low spending = saver
    Income_Spending_Ratio = Annual_Income / Spending_Score,

    # Age Group Buckets
    Age_Group = case_when(
      Age < 25             ~ "Young Adult (18-24)",
      Age >= 25 & Age < 35 ~ "Adult (25-34)",
      Age >= 35 & Age < 50 ~ "Middle Age (35-49)",
      Age >= 50            ~ "Senior (50+)"
    ),

    # Income Tier — thresholds updated to reflect actual USD values
    # (dataset range: $20,022 - $199,974)
    Income_Tier = case_when(
      Annual_Income < 80000  ~ "Low Income",
      Annual_Income < 140000 ~ "Mid Income",
      Annual_Income >= 140000 ~ "High Income"
    ),

    # Spending Category
    Spending_Category = case_when(
      Spending_Score < 35  ~ "Low Spender",
      Spending_Score < 65  ~ "Moderate Spender",
      Spending_Score >= 65 ~ "High Spender"
    )
  )

cat("✅ New features created:\n")
cat("  - Income_Spending_Ratio\n")
cat("  - Age_Group\n")
cat("  - Income_Tier\n")
cat("  - Spending_Category\n")

# Update scaled dataframe with engineered numeric feature
df_scaled <- df_scaled %>%
  mutate(Income_Spending_Ratio = scale(df$Income_Spending_Ratio))


# ============================================================
# STEP 5 — EXPLORATORY DATA ANALYSIS (EDA)
# ============================================================
```

```
cat("\n--- STEP 5: EXPLORATORY DATA ANALYSIS ---\n")

# --- 5.1 Gender Distribution ---
p1 <- ggplot(df, aes(x = Gender, fill = Gender)) +
  geom_bar(color = "white", width = 0.6) +
  scale_fill_manual(values = c("#E07B8A", "#5B8DB8")) +
  labs(title = "Gender Distribution", x = "Gender", y = "Count") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

p1

# --- 5.2 Age Distribution ---
p2 <- ggplot(df, aes(x = Age)) +
  geom_histogram(binwidth = 5, fill = "#5B8DB8", color = "white") +
  labs(title = "Age Distribution", x = "Age", y = "Frequency") +
  theme_minimal(base_size = 13)

p2

# --- 5.3 Annual Income Distribution ---
p3 <- ggplot(df, aes(x = Annual_Income)) +
  geom_histogram(binwidth = 10000, fill = "#4CAF84", color = "white") +
  scale_x_continuous(
    labels = scales::comma,
    breaks = seq(0, 250000, by = 50000)
  ) +
  labs(
    title = "Annual Income Distribution",
    x     = "Annual Income (USD)",
    y     = "Frequency"
  ) +
  theme_minimal(base_size = 13)

p3

# --- 5.4 Spending Score Distribution ---
p4 <- ggplot(df, aes(x = Spending_Score)) +
  geom_histogram(binwidth = 10, fill = "#E07B8A", color = "white") +
  labs(title = "Spending Score Distribution", x = "Spending Score (1-100)", y =
"Frequency") +
  theme_minimal(base_size = 13)

p4

# Display all 4 together
grid.arrange(p1, p2, p3, p4, ncol = 2)

# --- 5.5 Spending Score by Gender ---
p5 <- ggplot(df, aes(x = Gender, y = Spending_Score, fill = Gender)) +
  geom_boxplot(alpha = 0.7, color = "gray30") +
  scale_fill_manual(values = c("#E07B8A", "#5B8DB8")) +
  labs(title = "Spending Score by Gender", x = "Gender", y = "Spending Score") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

p5

# --- 5.6 Income vs Spending Scatterplot ---
p6 <- ggplot(df, aes(x = Annual_Income, y = Spending_Score, color = Gender)) +
  geom_point(size = 2.5, alpha = 0.7) +
  scale_color_manual(values = c("#E07B8A", "#5B8DB8")) +
  scale_x_continuous(
    labels = scales::comma,
    breaks = seq(0, 250000, by = 50000)
```

```
  ) +
  labs(
    title = "Annual Income vs Spending Score",
    x     = "Annual Income (USD)",
    y     = "Spending Score"
  ) +
  theme_minimal(base_size = 13)

p6

grid.arrange(p5, p6, ncol = 2)

# --- 5.7 Age Group Counts ---
p7 <- ggplot(df, aes(x = Age_Group, fill = Age_Group)) +
  geom_bar(color = "white") +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Customer Count by Age Group", x = "Age Group", y = "Count") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 15, hjust = 1))

p7

# --- 5.8 Correlation Matrix ---
numeric_df <- df %>% select(Age, Annual_Income, Spending_Score, Income_Spending_Ratio)
cor_matrix <- cor(numeric_df)

cat("\n📊 Correlation Matrix:\n")
print(round(cor_matrix, 2))

corrplot(cor_matrix,
         method = "color",
         type   = "upper",
         tl.col = "black",
         addCoef.col = "black",
         title  = "Correlation Matrix",
         mar    = c(0, 0, 1, 0))

cat("✅ EDA complete\n")


# ==============================================================
# STEP 6 — DETERMINE OPTIMAL NUMBER OF CLUSTERS
# ==============================================================

cat("\n--- STEP 6: DETERMINING OPTIMAL CLUSTERS ---\n")

# Use only the 2 most important features for classic Mall Customer segmentation
# (Annual Income + Spending Score — the "famous" 5-cluster result)
cluster_data <- df_scaled %>%
  select(Annual_Income, Spending_Score) %>%
  as.data.frame()

# --- 6.1 Elbow Method (WSS) ---
set.seed(123)
fviz_nbclust(cluster_data,
             kmeans,
             method = "wss",
             k.max  = 10,
             linecolor = "#5B8DB8") +
  labs(title = "Elbow Method — Optimal K",
       subtitle = "Look for the 'elbow' bend in the curve") +
  theme_minimal(base_size = 13)

# --- 6.2 Silhouette Method ---
```

```
fviz_nbclust(cluster_data,
             kmeans,
             method = "silhouette",
             k.max  = 10,
             linecolor = "#4CAF84") +
  labs(title = "Silhouette Method — Optimal K",
       subtitle = "Higher silhouette = better cluster separation") +
  theme_minimal(base_size = 13)

# --- 6.3 Gap Statistic ---
set.seed(123)
gap_stat <- clusGap(cluster_data,
                    FUN     = kmeans,
                    nstart  = 25,
                    K.max   = 10,
                    B       = 50)

fviz_gap_stat(gap_stat) +
  labs(title = "Gap Statistic — Optimal K") +
  theme_minimal(base_size = 13)

cat("\n📋 Gap Statistic Summary:\n")
print(gap_stat, method = "firstmax")

# --- 6.4 Hopkins Statistic (Cluster Tendency Check) ---
# Values close to 1 = strong clustering tendency

# DECISION: Based on Elbow + Silhouette, K = 5 is optimal for this dataset
optimal_k <- 5
cat("\n✅ Optimal K selected:", optimal_k, "\n")

# Instead of using full dataset size, sample 50 rows max
set.seed(123)
hopkins_val <- hopkins(cluster_data, n = 50)
cat("\n🔬 Hopkins Statistic:", round(hopkins_val$H, 3),
    "\n  (Values > 0.75 confirm strong cluster structure)\n")

# ==============================================================
# STEP 7 — CLUSTERING MODEL
# ==============================================================

optimal_k <- 5

cat("\n--- STEP 7: CLUSTERING MODEL (K-MEANS) ---\n")

set.seed(123)
kmeans_model <- kmeans(
  cluster_data,
  centers = optimal_k,
  nstart  = 25,      # Run 25 times with random starts, pick best
  iter.max = 100     # Maximum iterations
)

# Attach cluster labels back to original dataframe
df$Cluster <- as.factor(kmeans_model$cluster)

cat("✅ K-Means model fitted\n")
cat("\n📊 Cluster Sizes:\n")
print(table(df$Cluster))

cat("\n📍 Cluster Centers (Scaled):\n")
print(round(kmeans_model$centers, 3))

cat("\n📐 Within-cluster Sum of Squares:", round(kmeans_model$tot.withinss, 2), "\n")
```

```
cat("📐 Between-cluster Sum of Squares:", round(kmeans_model$betweenss, 2), "\n")
cat("📐 Total Sum of Squares:", round(kmeans_model$totss, 2), "\n")
cat("📐 BSS/TSS Ratio (higher = better):",
    round(kmeans_model$betweenss / kmeans_model$totss * 100, 1), "%\n")

# Silhouette score for our chosen K
sil <- silhouette(kmeans_model$cluster, dist(cluster_data))
cat("📐 Average Silhouette Width:", round(mean(sil[, 3]), 3),
    "\n  (Values > 0.5 = reasonable structure)\n")


# ============================================================
# STEP 8 — CLUSTER PROFILING + BUSINESS LABELING
# ============================================================

cat("\n--- STEP 8: CLUSTER PROFILING + BUSINESS LABELING ---\n")

# --- 8.1 Summary Statistics per Cluster ---
cluster_profile <- df %>%
  group_by(Cluster) %>%
  summarise(
    Count          = n(),
    Pct            = round(n() / nrow(df) * 100, 1),
    Avg_Age        = round(mean(Age), 1),
    Avg_Income     = round(mean(Annual_Income), 1),
    Avg_Spending   = round(mean(Spending_Score), 1),
    Pct_Female     = round(mean(Gender == "Female") * 100, 1),
    .groups        = "drop"
  )

cat("\n📋 CLUSTER PROFILE SUMMARY:\n")
print(cluster_profile)

# --- 8.2 Assign Business Labels ---
# Based on USD income ranges ($20k-$200k) and spending score (1-100)
# Inspect your cluster centers printout above and adjust if order differs
cluster_labels <- c(
  "1" = "High Income, Low Spender (Careful Savers)",
  "2" = "High Income, High Spender (Premium Targets)",
  "3" = "Mid Income, Mid Spender (Balanced Customers)",
  "4" = "Low Income, High Spender (Impulsive Buyers)",
  "5" = "Low Income, Low Spender (Budget Conscious)"
)

df$Cluster_Label <- cluster_labels[as.character(df$Cluster)]

cat("\n🏷️ CLUSTER BUSINESS LABELS:\n")
for (i in 1:optimal_k) {
  cat("Cluster", i, "→", cluster_labels[as.character(i)], "\n")
}

# --- 8.3 Detailed Profile per Cluster ---
cat("\n📊 DETAILED CLUSTER PROFILES:\n")
detailed_profile <- df %>%
  group_by(Cluster, Cluster_Label) %>%
  summarise(
    N              = n(),
    Avg_Age        = round(mean(Age), 1),
    Avg_Income     = round(mean(Annual_Income), 1),
    Avg_Spending   = round(mean(Spending_Score), 1),
    Pct_Female     = round(mean(Gender == "Female") * 100, 1),
    Pct_YoungAdult = round(mean(Age < 35) * 100, 1),
    .groups        = "drop"
  )
```

```
print(detailed_profile)


# ==============================================================
# STEP 9 — VISUALIZATION
# ==============================================================

cat("\n--- STEP 9: VISUALIZATION ---\n")

cluster_colors <- c("#E07B8A", "#5B8DB8", "#4CAF84", "#F4A261", "#9B59B6")

# --- 9.1 Classic Cluster Plot (Income vs Spending) ---
fviz_cluster(
  kmeans_model,
  data       = cluster_data,
  palette    = cluster_colors,
  geom       = "point",
  ellipse.type = "convex",
  ggtheme    = theme_minimal(base_size = 13),
  main       = "Customer Segments — Income vs Spending Score"
)

# --- 9.2 Scatter Plot with Labels ---
p_scatter <- ggplot(df, aes(x = Annual_Income, y = Spending_Score,
                            color = Cluster, shape = Gender)) +
  geom_point(size = 3, alpha = 0.8) +
  scale_color_manual(values = cluster_colors) +
  scale_x_continuous(labels = scales::comma) +
  labs(
    title    = "Customer Segmentation: Annual Income vs Spending Score",
    subtitle = "5 Distinct Customer Segments Identified",
    x        = "Annual Income (USD)",
    y        = "Spending Score (1-100)",
    color    = "Cluster",
    shape    = "Gender"
  ) +
  theme_minimal(base_size = 13)

print(p_scatter)

# --- 9.3 Cluster Size Bar Chart ---
p_bar <- ggplot(df, aes(x = Cluster, fill = Cluster)) +
  geom_bar(color = "white", width = 0.7) +
  scale_fill_manual(values = cluster_colors) +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.5, fontface = "bold") +
  labs(title = "Number of Customers per Cluster",
       x = "Cluster", y = "Count") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

print(p_bar)

# --- 9.4 Cluster Profiles Heatmap ---
profile_long <- cluster_profile %>%
  select(Cluster, Avg_Age, Avg_Income, Avg_Spending) %>%
  pivot_longer(-Cluster, names_to = "Metric", values_to = "Value")

p_heatmap <- ggplot(profile_long, aes(x = Metric, y = Cluster, fill = Value)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = round(Value, 1)), color = "white", fontface = "bold") +
  scale_fill_gradient(low = "#5B8DB8", high = "#E07B8A") +
  labs(title = "Cluster Profile Heatmap",
       x = "Metric", y = "Cluster") +
  theme_minimal(base_size = 13)
```

```
print(p_heatmap)

# --- 9.5 Silhouette Plot ---
fviz_silhouette(sil,
                palette    = cluster_colors,
                ggtheme    = theme_minimal(base_size = 12),
                main       = "Silhouette Plot — Cluster Quality")

# --- 9.6 Age Group Distribution per Cluster ---
p_age <- ggplot(df, aes(x = Cluster, fill = Age_Group)) +
  geom_bar(position = "fill", color = "white") +
  scale_fill_brewer(palette = "Set2") +
  scale_y_continuous(labels = percent_format()) +
  labs(title = "Age Group Composition per Cluster",
       x = "Cluster", y = "Proportion", fill = "Age Group") +
  theme_minimal(base_size = 13)

print(p_age)

cat("✅ All visualizations created\n")


# ==============================================================
# STEP 10 — BUSINESS RECOMMENDATIONS
# ==============================================================

cat("\n--- STEP 10: BUSINESS RECOMMENDATIONS ---\n")

cat("
╔═══════════════════════════════════════════════════════════════╗
║           BUSINESS RECOMMENDATIONS BY SEGMENT                 ╖
╚═══════════════════════════════════════════════════════════════╝

🔴 CLUSTER 1 — Careful Savers (High Income, Low Spender)
   Strategy: Build trust & justify value
   → Offer premium loyalty programs with clear benefits
   → Use data-driven personalization to show ROI of purchases
   → Focus on quality messaging, not discounts

🔵 CLUSTER 2 — Premium Targets (High Income, High Spender)
   Strategy: Retain & maximize lifetime value
   → VIP membership programs and exclusive early access
   → Upsell premium & luxury product lines
   → Personalized concierge-style customer service

🟢 CLUSTER 3 — Balanced Customers (Mid Income, Mid Spender)
   Strategy: Nurture and grow
   → Bundle deals and value-for-money offers
   → Loyalty point systems to increase purchase frequency
   → Targeted promotions based on purchase history

🟡 CLUSTER 4 — Impulsive Buyers (Low Income, High Spender)
   Strategy: Capture and convert quickly
   → Flash sales, limited-time offers, and seasonal promotions
   → Affordable payment plans and buy-now-pay-later options
   → Social media and influencer marketing

⚪ CLUSTER 5 — Budget Conscious (Low Income, Low Spender)
   Strategy: Attract with value
   → Highlight discounts, clearance, and everyday low prices
   → Focus on necessities and essentials
   → Win-back campaigns during sale seasons
")
```

```
# ============================================================
# STEP 11 — EXPORT RESULTS
# ============================================================

cat("\n--- STEP 11: EXPORTING RESULTS ---\n")

# --- 11.1 Export Segmented Customer Data ---
write.csv(df, "customer_segments_output.csv", row.names = FALSE)
cat("✅ Segmented data exported: customer_segments_output.csv\n")

# --- 11.2 Export Cluster Profile Summary ---
write.csv(cluster_profile, "cluster_profile_summary.csv", row.names = FALSE)
cat("✅ Cluster profiles exported: cluster_profile_summary.csv\n")

# --- 11.3 Save ALL Visualizations ---

# EDA Plots
ggsave("plot_01_gender_distribution.png",    plot = p1,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_01_gender_distribution.png\n")

ggsave("plot_02_age_distribution.png",       plot = p2,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_02_age_distribution.png\n")

ggsave("plot_03_income_distribution.png",    plot = p3,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_03_income_distribution.png\n")

ggsave("plot_04_spending_distribution.png",  plot = p4,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_04_spending_distribution.png\n")

ggsave("plot_05_spending_by_gender.png",     plot = p5,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_05_spending_by_gender.png\n")

ggsave("plot_06_income_vs_spending.png",     plot = p6,
       width = 10, height = 6, dpi = 300)
cat("✅ Saved: plot_06_income_vs_spending.png\n")

ggsave("plot_07_age_group_counts.png",       plot = p7,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_07_age_group_counts.png\n")

# EDA Combined Grid
png("plot_08_eda_combined_grid.png", width = 1600, height = 1200, res = 150)
grid.arrange(p1, p2, p3, p4, ncol = 2)
dev.off()
cat("✅ Saved: plot_08_eda_combined_grid.png\n")

# Clustering Plots
ggsave("plot_09_cluster_segments.png",       plot = p_scatter,
       width = 10, height = 7, dpi = 300)
cat("✅ Saved: plot_09_cluster_segments.png\n")

ggsave("plot_10_cluster_sizes.png",          plot = p_bar,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_10_cluster_sizes.png\n")

ggsave("plot_11_cluster_heatmap.png",        plot = p_heatmap,
```

```
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_11_cluster_heatmap.png\n")

ggsave("plot_12_age_per_cluster.png",        plot = p_age,
       width = 8, height = 5, dpi = 300)
cat("✅ Saved: plot_12_age_per_cluster.png\n")

# Correlation Matrix
png("plot_13_correlation_matrix.png", width = 800, height = 800, res = 150)
corrplot(cor_matrix,
         method      = "color",
         type        = "upper",
         tl.col      = "black",
         addCoef.col = "black",
         title       = "Correlation Matrix",
         mar         = c(0, 0, 1, 0))
dev.off()
cat("✅ Saved: plot_13_correlation_matrix.png\n")

# Silhouette Plot
png("plot_14_silhouette.png", width = 1000, height = 600, res = 150)
print(fviz_silhouette(sil,
                      palette = cluster_colors,
                      ggtheme = theme_minimal(base_size = 12),
                      main    = "Silhouette Plot — Cluster Quality"))
dev.off()
cat("✅ Saved: plot_14_silhouette.png\n")

# Elbow Method
png("plot_15_elbow_method.png", width = 1000, height = 600, res = 150)
print(fviz_nbclust(cluster_data, kmeans, method = "wss", k.max = 10,
                   linecolor = "#5B8DB8") +
        labs(title    = "Elbow Method — Optimal K",
             subtitle = "Look for the 'elbow' bend in the curve") +
        theme_minimal(base_size = 13))
dev.off()
cat("✅ Saved: plot_15_elbow_method.png\n")

# Silhouette Method
png("plot_16_silhouette_method.png", width = 1000, height = 600, res = 150)
print(fviz_nbclust(cluster_data, kmeans, method = "silhouette", k.max = 10,
                   linecolor = "#4CAF84") +
        labs(title    = "Silhouette Method — Optimal K",
             subtitle = "Higher silhouette = better cluster separation") +
        theme_minimal(base_size = 13))
dev.off()
cat("✅ Saved: plot_16_silhouette_method.png\n")

# Gap Statistic
png("plot_17_gap_statistic.png", width = 1000, height = 600, res = 150)
print(fviz_gap_stat(gap_stat) +
        labs(title = "Gap Statistic — Optimal K") +
        theme_minimal(base_size = 13))
dev.off()
cat("✅ Saved: plot_17_gap_statistic.png\n")

# Classic Cluster Plot
png("plot_18_classic_cluster_plot.png", width = 1000, height = 700, res = 150)
print(fviz_cluster(kmeans_model,
                   data         = cluster_data,
                   palette      = cluster_colors,
                   geom         = "point",
                   ellipse.type = "convex",
                   ggtheme      = theme_minimal(base_size = 13),
```

```
                    main         = "Customer Segments — Income vs Spending Score"))
dev.off()
cat("✅ Saved: plot_18_classic_cluster_plot.png\n")

# --- 11.4 Confirm All Files Saved ---
cat("\n📁 ALL EXPORTED FILES:\n")
exported_files <- list.files(pattern = "\\.(csv|png)$")
for (f in exported_files) {
  cat(" →", f, "\n")
}

# --- 11.5 Session Info (for reproducibility) ---
cat("\n📋 SESSION INFO:\n")
sessionInfo()

cat("\n
╔═══════════════════════════════════════════════════════════╗
   ✅ PROJECT 1 COMPLETE — CUSTOMER SEGMENTATION            ╚
   18 plots + 2 CSV files saved to your working directory
   Ready for GitHub upload
\n")


getwd()
```